WEBRUM-06: Performance Analysis

> **Series:** WEBRUM — Web Real User Monitoring | **Notebook:** 6 of 10 | **Created:** March 2026 | **Last Updated:** 08/12/2026

Overview

Web performance directly impacts user experience, conversion rates, and SEO rankings. While Core Web Vitals (covered in WEBRUM-03) provide high-level scoring, a deeper performance analysis requires understanding the full page load waterfall — from DNS lookup to load complete — and how performance varies across geographies, network conditions, and device types.

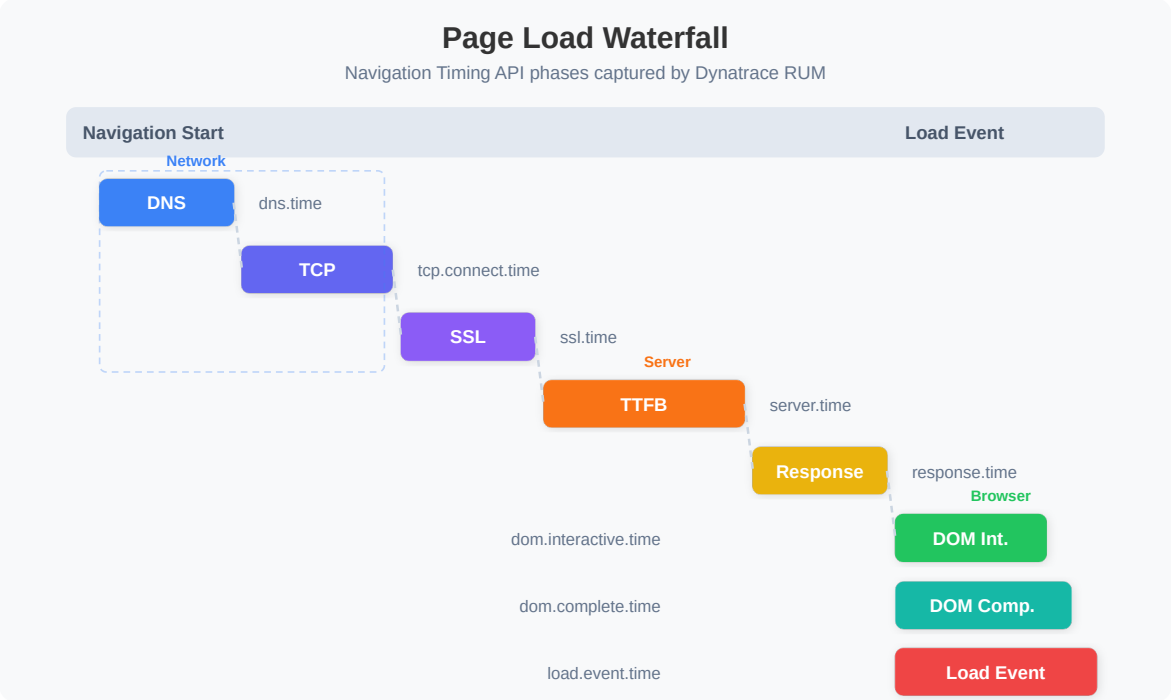
Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
RUM Enabled	Web applications with detailed timing capture
Permissions	`storage:events:read`
Previous Notebooks	WEBRUM-01: RUM Fundamentals, WEBRUM-03: Core Web Vitals

1. Page Load Waterfall

Every page load follows a sequence of phases captured by the browser's Navigation Timing API. Dynatrace records these milestones for each load action:



Timing Milestones Explained

Milestone	Description	Optimization Focus
DNS time	Domain name resolution	Use DNS prefetching, reduce DNS lookups
TCP connect	Establishing TCP connection	Enable HTTP/2, use CDN edge servers
SSL time	TLS handshake	Enable TLS 1.3, OCSP stapling
TTFB	Server processing + first byte transit	Optimize server response time
Response time	Full HTML download	Compress, minimize HTML size
DOM interactive	HTML parsed, DOM ready for interaction	Defer non-critical JS, reduce blocking resources
DOM complete	All sub-resources loaded	Lazy load images, async load scripts
Load event	`window.onload` fires	Final milestone; all resources ready

```
// Error / navigation vocabulary corrected 08/12/2026 (New RUM):
//   filter type == "Error"  -&gt; filter characteristics.has_error == true
(11,909 events; identical
//                              population to isNotNull(error.type), whose
values are request/csp/exception)
//   error.message           -&gt; error.reason
//   user_action.type == "RouteChange" -&gt; "same_view"  (the New RUM SPA
route-change value; the
//                              only other value is "hard_navigation".
"Custom" has NO equivalent.)
//   connection.type         -&gt; network.protocol.name
// CLASSIFIER MATTERS AS MUCH AS THE FIELD: navigation-timing fields
(performance.dom_interactive,
// performance.load_event_end) live on classifier "navigation" and are 0 on
"page_summary", so a
// page_summary filter silently empties them. ttfb.* is the opposite — it
lives on page_summary.
// Session/performance field vocabulary corrected 08/12/2026 (New RUM).
Classic camelCase RUM
// names are null on New RUM data and fail silently. Verified against 3,261
user.sessions:
//   userType -&gt; dt.rum.user_type      userActionCount -&gt;
user_action_count
//   totalErrorCount -&gt; error.count    sessionId -&gt; dt.rum.session.id
//   hasSessionReplay -&gt; characteristics.has_replay
//   browserFamily -&gt; browser.name     osFamily -&gt; os.name
//   application -&gt; primary_tags.application
//   country/city/continent -&gt; geo.country.name / geo.city.name /
geo.continent.name
//   screen.width|height -&gt; browser.window.width|height
//   dom.interactive.time -&gt; performance.dom_interactive
//   load.event.time -&gt; performance.load_event_end
//   server.time -&gt; ttfb.waiting_duration
// TWO TENANT CAVEATS on the validation tenant, both of which leave a CORRECT
query empty:
//   * every session is dt.rum.user_type == "synthetic", so a real-user filter
matches nothing —
//     the "real_user" literal itself could NOT be confirmed here and is the
documented value form;
//   * geo.* is 0-populated, because synthetic traffic carries no geolocation.
// Field vocabulary corrected 08/12/2026 — this series targets **New RUM**,
but was written
// against names that are null on New RUM data, so these cells returned
nothing while erroring
// nowhere. Verified against 5,556,127 user.events records (schema 0.24.0,
javascript agent):
//   action.type == "Load"             -&gt; characteristics.classifier ==
```

```
"navigation"
//   action.type                        -&gt; user_action.type
(hard_navigation | same_view)
//   action.name                        -&gt; page.detected_name
//   web_vitals.largest_contentful_paint-&gt; lcp.start_time       (327,099
populated)
//   web_vitals.cumulative_layout_shift -&gt; cls.value            (387,254
populated)
//   app.name                           -&gt; dt.rum.application.id
// UNITS CHANGE WITH THE FIELD. web_vitals.* was a nanosecond DURATION, so `/
1ms` was correct for
// it; lcp.start_time is a PLAIN NUMBER already in milliseconds, and dividing
it by 1ms yields
// null. Compare it against the 2500/4000 ms thresholds directly.
// `web_vitals.*` does still exist in the same schema, but carried 16 records
in 30 days against
// lcp.*'s 327,099 — it is not a different RUM generation, just a rarely-
populated sibling.
// Page load waterfall — average timing breakdown for top 10 pages
fetch user.events, from:-24h
| filter characteristics.classifier == "navigation"
| summarize page_views = count(),
    avg_duration = avg(duration),
    avg_dom_interactive = avg(performance.dom_interactive),
    avg_load_event = avg(performance.load_event_end),
    avg_server_time = avg(ttfb.waiting_duration),
    by:{page.detected_name}
| sort page_views desc
| limit 10
```

2. Time to First Byte (TTFB)

TTFB measures the time from the browser sending the request to receiving the first byte of the response. It reflects:

- **Server processing time** — How long the backend takes to generate the response
- **Network latency** — Round-trip time between client and server
- **CDN performance** — Cache hit/miss at the edge

Google recommends a TTFB of **≤ 800ms** for a good user experience.

```
// TTFB analysis by page — identify pages with slow server response
fetch user.events, from:-24h
| filter characteristics.classifier == "page_summary"
| filter isNotNull(ttfb.waiting_duration)
| summarize page_views = count(),
    avg_ttfb = avg(ttfb.waiting_duration),
    p75_ttfb = percentile(ttfb.waiting_duration, 75),
    p95_ttfb = percentile(ttfb.waiting_duration, 95),
    by:{page.detected_name}
| filter page_views &gt; 10
| sort p75_ttfb desc
| limit 10
```

```
// Unit trap (08/12/2026): New RUM timing fields such as lcp.start_time and
ttfb.waiting_duration
// are PLAIN NUMBERS already in milliseconds, not durations. `field / 1ms`
yields null on them —
// silently, so a chart of nulls looks like "no data". Compare and aggregate
them directly.
// TTFB distribution — classify into Good / Needs Improvement / Poor
fetch user.events, from:-24h
| filter characteristics.classifier == "page_summary"
| filter isNotNull(ttfb.waiting_duration)
| fieldsAdd ttfb_ms = ttfb.waiting_duration
| fieldsAdd ttfb_category = if(ttfb_ms &lt;= 800, "Good",
    else: if(ttfb_ms &lt;= 1800, "Needs Improvement",
    else: "Poor"))
| summarize action_count = count(), by:{ttfb_category}
| sort action_count desc
```

3. DOM Interactive and Load Event

DOM Interactive is when the HTML has been fully parsed and the DOM tree is ready — but images, stylesheets, and sub-frames may still be loading. This is when users can first interact with the page.

Load Event fires when all resources (images, scripts, stylesheets, iframes) have finished loading. The gap between DOM Interactive and Load Event reveals how much time is spent on sub-resource loading.

```
// DOM Interactive vs Load Event — identify resource-heavy pages
fetch user.events, from:-24h
| filter characteristics.classifier == "navigation"
| filter isNotNull(performance.dom_interactive) and
isNotNull(performance.load_event_end)
| summarize page_views = count(),
    avg_dom_interactive = avg(performance.dom_interactive),
    avg_load_event = avg(performance.load_event_end),
    by:{page.detected_name}
| fieldsAdd resource_load_gap = avg_load_event - avg_dom_interactive
| filter page_views &gt; 10
| sort resource_load_gap desc
| limit 10
```

A large gap between DOM Interactive and Load Event suggests the page has many sub-resources (large images, heavy scripts, third-party tags) that delay full load completion. Consider lazy loading and async script loading to reduce this gap.

4. Performance by Geography

Performance varies significantly by user location due to network latency, CDN coverage, and server proximity.

```
// Page load performance by country — identify slow regions
fetch user.events, from:-24h
| filter characteristics.classifier == "page_summary"
| filter isNotNull(geo.country.name)
| summarize page_views = count(),
    avg_duration_ms = avg(duration / 1ms),
    p75_duration_ms = percentile(duration / 1ms, 75),
    avg_ttfb_ms = avg(ttfb.waiting_duration),
    by:{geo.country.name}
| filter page_views &gt; 20
| sort p75_duration_ms desc
| limit 15
```

```
// Compare TTFB across regions — CDN effectiveness indicator
fetch user.events, from:-24h
| filter characteristics.classifier == "page_summary"
| filter isNotNull(geo.continent.name)
| summarize page_views = count(),
    avg_ttfb_ms = avg(ttfb.waiting_duration),
    p75_ttfb_ms = percentile(ttfb.waiting_duration, 75),
    by:{geo.continent.name}
| sort p75_ttfb_ms desc
```

> **Tip:** If TTFB is consistently high for specific regions, consider deploying CDN edge servers or regional server instances closer to those users.

5. Performance by Network and Device

Network connection type and device capability significantly impact perceived performance.

```
// Performance by connection type — wifi vs cellular vs wired
fetch user.events, from:-24h
| filter characteristics.classifier == "page_summary"
| filter isNotNull(network.protocol.name)
| summarize page_views = count(),
    avg_duration_ms = avg(duration / 1ms),
    p75_duration_ms = percentile(duration / 1ms, 75),
    by:{network.protocol.name}
| sort p75_duration_ms desc
```

```
// Performance by browser — which browsers are slowest?
fetch user.events, from:-24h
| filter characteristics.classifier == "page_summary"
| filter isNotNull(browser.name)
| summarize page_views = count(),
    avg_duration_ms = avg(duration / 1ms),
    p75_duration_ms = percentile(duration / 1ms, 75),
    by:{browser.name}
| filter page_views &gt; 20
| sort p75_duration_ms desc
| limit 10
```

```
// Performance by OS — desktop vs mobile operating systems
fetch user.events, from:-24h
| filter characteristics.classifier == "page_summary"
| filter isNotNull(os.family)
| summarize page_views = count(),
    avg_duration_ms = avg(duration / 1ms),
    p75_duration_ms = percentile(duration / 1ms, 75),
    by:{os.family}
| filter page_views &gt; 20
| sort p75_duration_ms desc
```

6. Identifying Slow Pages

Find the pages that need optimization attention — ranked by the impact of their slowness (volume x duration).

```
// Slowest pages by p95 duration — worst-case performance
fetch user.events, from:-24h
| filter characteristics.classifier == "page_summary"
| summarize page_views = count(),
    avg_ms = avg(duration / 1ms),
    p75_ms = percentile(duration / 1ms, 75),
    p95_ms = percentile(duration / 1ms, 95),
    by:{page.detected_name}
| filter page_views &gt; 20
| sort p95_ms desc
| limit 10
```

```
// Weighted impact score — pages with high traffic AND high duration
fetch user.events, from:-24h
| filter characteristics.classifier == "page_summary"
| summarize page_views = count(),
    avg_ms = avg(duration / 1ms),
    by:{page.detected_name}
| fieldsAdd impact_score = page_views * avg_ms
| sort impact_score desc
| limit 10
```

> **Tip:** The impact score (page views x average duration) helps prioritize optimization efforts. A moderately slow page with high traffic may be more impactful than a very

> slow page with few visitors.

7. Performance Trends

Track performance over time to detect regressions and measure the impact of optimizations.

```
// Page load duration trend — daily p75 over the last 7 days
fetch user.events, from:-7d
| filter characteristics.classifier == "page_summary"
| fieldsAdd duration_ms = duration / 1ms
| makeTimeseries p75_duration = percentile(duration_ms, 75), interval:1d
```

```
// TTFB trend — hourly p75 over the last 24 hours
fetch user.events, from:-24h
| filter characteristics.classifier == "page_summary"
| filter isNotNull(ttfb.waiting_duration)
| fieldsAdd ttfb_ms = ttfb.waiting_duration
| makeTimeseries p75_ttfb = percentile(ttfb_ms, 75), interval:1h
```

8. Summary and Next Steps

In this notebook, we covered:

- **Page load waterfall** — Full timing breakdown from DNS to load complete
- **TTFB analysis** — Server response time measurement and classification
- **DOM timing** — Interactive vs complete timing for resource load gap analysis
- **Geographic performance** — Regional and continental performance differences
- **Network/device performance** — Impact of connection type, browser, and OS
- **Slow page identification** — Ranking pages by p95 duration and impact score
- **Performance trends** — Time-series tracking for regression detection

Next Steps

- **WEBRUM-07: Session Replay** — Visually investigate slow page experiences

- **WEBRUM-08: Dashboards and Alerting** — Build operational RUM dashboards with Apdex

References

- Dynatrace Performance Analysis
- Navigation Timing API
- TTFB Best Practices
